شرح سطر بسطر ١٩ — فهرس التعلّم والمزوّدات وخطة المراجعة

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

ثلاث قطع تربط البيانات بالشاشات: `LearningIndex` (لقطة صرفة من تقدّم الطالب تجيب «ما حالة هذا الدرس؟»)، ومزوّدات Riverpod التي تغذّيه من القاعدة، و`ReviewPlanner` الذي يبني خطة مراجعة قصيرة لأضعف مفهوم.

المبدأ الدائم: لا نخزّن ما يمكن حسابه. حالة الدرس ونسبته والنجوم تُحسب من الأجزاء المنجزة والمحاولات، فلا تخرج عن التزامن.

lib/features/learning/learning_index.dart — فهرس التقدّم

lib/features/learning/learning_index.dart

الأسطر 2–1

```
import '../../core/db/app_database.dart';
import '../catalog/catalog_models.dart';
```

ماذا يفعل؟ استيراد صفوف القاعدة ونماذج الكتالوج.

لماذا بهذه الطريقة؟ الفهرس يربط الاثنين.

الأسطر 13–4

```
/// لقطة من تقدّم الطالب في كل الدروس، تُبنى من صفوف القاعدة. دوالّها صرفة
/// (بلا قاعدة بيانات ولا واجهة) فتُختبر مباشرة.
class LearningIndex {
  LearningIndex({
    required Iterable<LessonProgressEntry> progress,
    required Iterable<LessonPartEntry> parts,
  }) : _progress = {for (final p in progress) p.lessonId: p},
       _parts = _groupParts(parts);

  const LearningIndex.empty() : _progress = const {}, _parts = const {};
```

ماذا يفعل؟ المُنشئ يفهرس تقدّم الدروس بمعرّف الدرس، ويجمع الأجزاء المنجزة لكل درس. و`.empty()` فهرس فارغ ثابت.

لماذا بهذه الطريقة؟ خرائط بالمعرّف تجعل كل سؤال O(1). الفهرس الفارغ حالة «قبل وصول البيانات» فتُرسم الشاشات بلا انتظار.

الأسطر 15–26

```
15    final Map<String, LessonProgressEntry> _progress;
16    final Map<String, Set<LessonPart>> _parts;
17
18    static Map<String, Set<LessonPart>> _groupParts(
19      Iterable<LessonPartEntry> parts,
20    ) {
21      final out = <String, Set<LessonPart>>{};
22      for (final p in parts) {
23        (out[p.lessonId] ??= <LessonPart>{}).add(p.part);
24      }
25      return out;
26    }
```

ماذا يفعل؟ الحقلان و`_groupParts`: يبني `Map<درس, Set<جزء>>` بعملية `??=`.

لماذا بهذه الطريقة؟ `??=` ينشئ المجموعة عند أول جزء للدرس ثم يضيف فلا نحتاج فحصًا منفصلًا.

الأسطر 28–40

```
28    LessonStatus statusOf(String lessonId) =>
29        _progress[lessonId]?.status ?? LessonStatus.notStarted;
30
31    /// تقدّم الدرس بين 0 و1 (الأجزاء المنجزة ÷ المتوفرة).
32    double progressOf(String lessonId) => _progress[lessonId]?.progress ?? 0;
33
34    Set<LessonPart> partsDone(String lessonId) =>
35        _parts[lessonId] ?? const <LessonPart>{};
36
37    DateTime? updatedAtOf(String lessonId) => _progress[lessonId]?.updatedAt;
38
39    bool isCompleted(String lessonId) =>
40        statusOf(lessonId) == LessonStatus.completed;
```

ماذا يفعل؟ حالة الدرس وتقدّمه والأجزاء المنجزة ووقت آخر تحديث، و`isCompleted`.

لماذا بهذه الطريقة؟ الدرس غير الموجود في القاعدة «لم يبدأ» وتقدّمه صفر: لا حاجة لصفوف فارغة لكل درس.

الأسطر 42–54

```
42    bool isStarted(String lessonId) =>
43        statusOf(lessonId) != LessonStatus.notStarted;
44
45    int completedCount(Iterable<CatalogLesson> lessons) =>
46        lessons.where((l) => isCompleted(l.id)).length;
47
48    /// نسبة تقدّم مجموعة دروس (وحدة أو مادة): المكتمل ÷ الكل.
49    double fraction(List<CatalogLesson> lessons) =>
50        lessons.isEmpty ? 0 : completedCount(lessons) / lessons.length;
51
52    /// هل بدأ الطالب أي درس من هذه المجموعة؟
53    bool startedAny(Iterable<CatalogLesson> lessons) =>
54        lessons.any((l) => isStarted(l.id));
```

ماذا يفعل؟ `isStarted` وعدد المكتمل ونسبة مجموعة دروس (وحدة/مادة) و`startedAny`.

لماذا بهذه الطريقة؟ نسبة الوحدة = المكتمل ÷ الكل، تُعرض في بطاقات الوحدات والمواد.

الأسطر 56–76

```
  /// الدرس الذي سيكمل منه الطالب في مجموعة: أحدث درس قيد التعلّم، وإلا أول
  /// درس لم يكتمل، وإلا null.
  CatalogLesson? resumeLesson(List<CatalogLesson> lessons) {
    CatalogLesson? best;
    DateTime? bestAt;
    for (final l in lessons) {
      if (statusOf(l.id) != LessonStatus.inProgress) continue;
      final at = updatedAtOf(l.id);
      if (best == null ||
          (at != null && (bestAt == null || at.isAfter(bestAt)))) {
        best = l;
        bestAt = at;
      }
    }
    if (best != null) return best;
    for (final l in lessons) {
      if (!isCompleted(l.id)) return l;
    }
    return null;
  }
}
```

ماذا يفعل؟ `resumeLesson`: أحدث درس قيد التعلّم، وإلا أول درس غير مكتمل، وإلا null.

لماذا بهذه الطريقة؟ زر «تابع» يفتح ما يتوقعه الطالب: الدرس الذي توقّف عنده.

lib/features/learning/learning_providers.dart — المزوّدات

lib/features/learning/learning_providers.dart

الأسطر 1–10

```
import 'dart:math';

import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/db/app_database.dart';
import '../../core/db/database_provider.dart';
import '../home/home_providers.dart';
import '../profiles/active_profile_provider.dart';
import 'learning_index.dart';
import 'learning_repository.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟ تشمل `dart:math` لمصدر العشوائية.

الأسطر 12–17

```dart
final learningRepositoryProvider = Provider<LearningRepository>((ref) {
  return LearningRepository(
    ref.watch(appDatabaseProvider),
    now: ref.watch(clockProvider),
  );
});
```

ماذا يفعل؟ مزوّد المستودع: يأخذ القاعدة والساعة.

لماذا بهذه الطريقة؟ الساعة تُحقن ليمكن اختبار الأوقات.

الأسطر 19–26

```dart
/// الأجزاء المنجزة في دروس طالب (تتحدّث تلقائيًا).
final lessonPartsProvider = StreamProvider.autoDispose
    .family<List<LessonPartEntry>, String>(
      (ref, profileId) => ref
          .watch(appDatabaseProvider)
          .activityDao
          .watchLessonParts(profileId),
    );
```

ماذا يفعل؟ مزوّد بثّ للأجزاء المنجزة لطالب.

لماذا بهذه الطريقة؟ `StreamProvider` يعيد بناء الواجهة تلقائيًا فور إنجاز جزء.

الأسطر 28–37

```dart
/// تقدّم الملف النشط في كل الدروس. قبل وصول البيانات (أو بلا ملف نشط) يكون
/// فارغًا: كل الدروس «لم تبدأ».
final learningIndexProvider = Provider.autoDispose<LearningIndex>((ref) {
  final id = ref.watch(activeProfileIdProvider).value;
  if (id == null) return const LearningIndex.empty();
  return LearningIndex(
    progress: ref.watch(lessonProgressProvider(id)).value ?? const [],
    parts: ref.watch(lessonPartsProvider(id)).value ?? const [],
  );
});
```

ماذا يفعل؟ `learningIndexProvider`: يبني الفهرس من تقدّم الدروس والأجزاء للملف النشط، وفارغ قبل التحميل أو بلا ملف.

لماذا بهذه الطريقة؟ كل شاشة تعلّم تأخذ فهرسًا جاهزًا بدل ربط مصدرين بنفسها.

الأسطر 39–40

```dart
/// مصدر العشوائية لخلط الأسئلة؛ يُستبدل بمصدر ثابت في الاختبارات.
final roundRandomProvider = Provider<Random>((ref) => Random());
```

ماذا يفعل؟ مزوّد العشوائية.

لماذا بهذه الطريقة؟ تُثبّته الاختبارات ببذرة لنتائج متوقعة.

الأسطر 42–48

```dart
/// مجموع نجوم الملف النشط (الرقم الذهبي في رأس الجولة).
final starsTotalProvider = Provider.autoDispose<int>((ref) {
  final id = ref.watch(activeProfileIdProvider).value;
  if (id == null) return 0;
  final attempts = ref.watch(attemptsProvider(id)).value ?? const [];
  return attempts.fold(0, (sum, a) => sum + a.starsEarned);
});
```

ماذا يفعل؟ مجموع نجوم الملف النشط من المحاولات.

لماذا بهذه الطريقة؟ النجوم تُحسب ولا تُخزَّن.

الأسطر 50–60

```dart
typedef LoadedAttempt = ({Attempt attempt, AttemptDetails details});

/// محاولة محفوظة بتفاصيلها (لشاشتي نهاية الجولة والنتيجة). null = غير موجودة.
final attemptDetailsProvider = FutureProvider.autoDispose
    .family<LoadedAttempt?, int>((ref, attemptId) async {
      final profileId = await ref.watch(activeProfileIdProvider.future);
      if (profileId == null) return null;
      return ref
          .watch(learningRepositoryProvider)
          .loadAttempt(profileId, attemptId);
    });
```

ماذا يفعل؟ نوع سجل `LoadedAttempt` ومزوّد محاولة بتفاصيلها.

لماذا بهذه الطريقة؟ شاشتا نهاية الجولة والنتيجة تقرآن من القاعدة بدل تمرير بيانات عبر المسار (التي تضيع عند إعادة فتح التطبيق).

lib/features/learning/review_plan.dart — خطة المراجعة

lib/features/learning/review_plan.dart

منطق صرف مُختبَر (8 اختبارات) بلا واجهة.

الأسطر 1–4

```dart
import '../../core/constants/learning_rules.dart';
import '../../app/routes.dart';
import '../../core/db/enums.dart';
import '../content/lesson_content.dart';
```

ماذا يفعل؟ الاستيرادات: المسارات والأجزاء والمحتوى.

لماذا بهذه الطريقة؟ الخطة تحمل مسار كل خطوة جاهزًا.

الأسطر 6–26

```dart
/// خطوة في خطة المراجعة.
class ReviewStep {
  const ReviewStep({
    required this.part,
    required this.route,
    required this.done,
    this.count = 0,
    this.minutes = 0,
  });

  /// الجزء الذي تفتحه الخطوة (شرح، فيديو، تدريب، اختبار).
  final LessonPart part;
  final String route;
  final bool done;

  /// عدد التمارين المخصّصة (خطوة التدريب).
  final int count;

  /// المدة التقريبية (شرح/فيديو).
  final int minutes;
}
```

ماذا يفعل؟ `ReviewStep`: الجزء والمسار وهل أُنجز وعدد التمارين والدقائق.

لماذا بهذه الطريقة؟ سجل بيانات للشاشة؛ لا منطق فيه.

الأسطر 28–49

```dart
/// خطة قصيرة لمفهوم واحد ضعيف: شرح ← فيديو ← تمارين مخصّصة ← إعادة الاختبار
class ReviewPlan {
  const ReviewPlan({
    required this.conceptId,
    required this.mastery,
    required this.steps,
  });

  final String conceptId;

  /// إتقان المفهوم الحالي (0 إن لم يُقَس بعد).
  final double mastery;
  final List<ReviewStep> steps;

  /// الخطوة الحالية: أول خطوة لم تُنجَز، أو null إن أُنجزت كلها.
  ReviewStep? get current {
    for (final s in steps) {
      if (!s.done) return s;
    }
    return null;
  }
}
```

ماذا يفعل؟ `ReviewPlan`: المفهوم والإتقان والخطوات، و`current` = أول خطوة غير منجزة.

لماذا بهذه الطريقة؟ زر «تابع» في الشاشة يفتح هذه الخطوة.

الأسطر 51–56

```dart
abstract final class ReviewPlanner {
  /// عتبة «الإتقان».
  static const target = kMasteryThreshold;

  /// أقصى تمارين في الخطوة المخصّصة.
  static const maxPractice = 5;
```

ماذا يفعل؟ ثوابت المخطِّط: عتبة الإتقان = (0.6) `kMasteryThreshold`، وأقصى تمارين 5.

لماذا بهذه الطريقة؟ كانت 0.8؛ صارت العتبة الموحّدة نفسها في الرئيسية والنتيجة، فالمفهوم الذي يظهر «يحتاج مراجعة» في مكان لا يُعدّ «متقنًا» في مكان آخر.

الأسطر 58–75

```dart
  /// أضعف مفهوم دون [target]، أو null إن كانت كل مفاهيم الدرس متقنة.
  /// (يحتاج تدريبًا) المفهوم الذي لم يُقَس بعد يُعدّ إتقانه صفرًا.
  static String? weakest(
    List<String> lessonConcepts,
    Map<String, double> mastery,
  ) {
    String? worst;
    var worstValue = 1.0;
    for (final id in lessonConcepts) {
      final v = mastery[id] ?? 0;
      if (v >= target) continue;
      if (worst == null || v < worstValue) {
        worst = id;
        worstValue = v;
      }
    }
    return worst;
  }
```

ماذا يفعل؟ `weakest`: أضعف مفهوم دون العتبة (غير المقيس = صفر)، ويعيد null إن كانت كلها متقنة. التعادل لأول مفهوم.

لماذا بهذه الطريقة؟ مفهوم لم يُجرَّب يحتاج تدريبًا قبل الحكم عليه متقنًا.

الأسطر 77–86

```dart
  static ReviewPlan? build({
    required String lessonId,
    required List<String> lessonConcepts,
    required Map<String, double> mastery,
    required LessonContent? content,
    required Set<LessonPart> partsDone,
  }) {
    final concept = weakest(lessonConcepts, mastery);
    if (concept == null || content == null) return null;
```

ماذا يفعل؟ `build`: يختار المفهوم ويعيد null لو لا ضعف أو لا محتوى.

لماذا بهذه الطريقة؟ الشاشة تعرض «أتقنت هذا الدرس» عند null.

الأسطر 87–97

```
87      final steps = <ReviewStep>[];
88      if (content.has(LessonPart.explanation)) {
89        steps.add(
90          ReviewStep(
91            part: LessonPart.explanation,
92            route: AppRoutes.lessonPart(lessonId, LessonPart.explanation.name),
93            done: partsDone.contains(LessonPart.explanation),
94            minutes: content.steps.length,
95          ),
96        );
97      }
```

ماذا يفعل؟ خطوة الشرح إن توفر (المدة = عدد الخطوات كدقائق).

لماذا بهذه الطريقة؟ نتحقق بـ `has` فلا تظهر خطوة لجزء غير موجود.

الأسطر 98–107

```
98      if (content.has(LessonPart.video)) {
99        steps.add(
100         ReviewStep(
101           part: LessonPart.video,
102           route: AppRoutes.lessonPart(lessonId, LessonPart.video.name),
103           done: partsDone.contains(LessonPart.video),
104           minutes: content.video!.minutes,
105         ),
106       );
107     }
```

ماذا يفعل؟ خطوة الفيديو إن توفر.

لماذا بهذه الطريقة؟ تُنجز بإنجاز جزء الفيديو.

الأسطر 108–119

```
108     if (content.has(LessonPart.practice)) {
109       final own = content.practice.where((q) => q.concept == concept).length;
110       final n = (own > 0 ? own : content.practice.length).clamp(1, maxPractice);
111       steps.add(
112         ReviewStep(
113           part: LessonPart.practice,
114           route: '${AppRoutes.round(lessonId)}?concept=$concept',
115           done: false,
116           count: n,
117         ),
118       );
119     }
```

ماذا يفعل؟ خطوة التمارين المخصّصة: عددها أسئلة المفهوم (أو كل الأسئلة إن لم يوجد) بحدّ 5، ومسارها فيه `?concept=` .

لماذا بهذه الطريقة؟ تعتمد على حصر المفهوم في المحرّك. لا تُعلَّم منجزة لأن نتيجتها تظهر في الإتقان نفسه.

الأسطر 120–128

```
    if (content.has(LessonPart.quiz)) {
      steps.add(
        ReviewStep(
          part: LessonPart.quiz,
          route: AppRoutes.quiz(lessonId),
          done: false,
        ),
      );
    }
```

ماذا يفعل؟ خطوة إعادة الاختبار إن توفر.

لماذا بهذه الطريقة؟ قياس التحسّن بعد المراجعة.

الأسطر 129–136

```
    if (steps.isEmpty) return null;
    return ReviewPlan(
      conceptId: concept,
      mastery: mastery[concept] ?? 0,
      steps: steps,
    );
  }
}
```

ماذا يفعل؟ بلا خطوات = لا خطة، وإلا بناء `ReviewPlan`.

لماذا بهذه الطريقة؟ خطة فارغة بلا معنى.